ECE180 Final Project

Srinivasan Arumugham, Rachit Gupta, Riya Gupta, Nikhil Kapasi, Ethan Solomon

June 16, 2024

Abstract

This report presents a comprehensive approach to analyzing and predicting Google stock prices using a deep neural network model based on a bidirectional Long Short-Term Memory (BiLSTM) architecture [SNTN19]. Our methodology involves a detailed exploration and preprocessing of historical stock data, emphasizing the significance of data normalization and sequence generation for effective model training. We experiment with various model architectures, including parallel LSTMs and BiLSTMs, and conduct an extensive hyperparameter search to optimize model performance. Our final BiLSTM model demonstrates robust predictive capabilities, achieving a validation mean squared error (MSE) of 0.000154 and a test MSE of 0.002587. This underscores the model's proficiency in capturing complex patterns in stock price movements, offering valuable insights for financial analysis and decision-making.

1 Introduction

Stock price prediction is a critical yet challenging task in the financial domain. Accurate forecasts can lead to significant financial gains, drawing interest from researchers, large banks, and corporations. Predicting stock prices involves analyzing historical data to identify patterns and trends. The flippant nature of the stock market necessitates modeling approaches attuned to such volatility, as traditional methods often fail to capture complex dependencies. This project aims to predict Google stock prices using a deep neural network model based on a bidirectional Long Short-Term Memory (BiLSTM) architecture.

Our methodology begins with a thorough exploration of the Google stock dataset, where we analyze the features of our data and consider how we might want to model it. We then design and train a BiLSTM model, iterating on various architectures and hyperparameters to enhance performance. This architecture was chosen for its ability to capture both short-term and long-term dependencies in sequential data by processing information in both forward and backward directions.

The following sections outline our methodology, including data exploration, preprocessing, and model design. We present our training strategy and model results, demonstrating its effectiveness in predicting Google stock prices.

2 Methods

2.1 Data Exploration

We started by using various visualization tactics to understand the data and develop an idea of what we wanted to do with our model. Once we plotted the data in `GOOG.csv`, We found that daily trading volume, the various values of the stock itself (open, close, high, low, etc), and the delta of these values were most important in highlighting trends and anomalies that would allow us to make informed decisions about our model design.

In Figure 1, we plotted the four data points (open, close, high, low) collected each day overlaid on top of one another. From this initial naive plot, we found that the closing stock price was the least volatile dataset. High and low tended to have larger daily fluctuations, while the opening stock price seemed to have general unpredictability during more "noisy" periods. Though all four datasets followed the same general trend, we decided to focus on the closing price, as we hoped the consistency in the data would help the model discern useful information about stock movement from the noise.

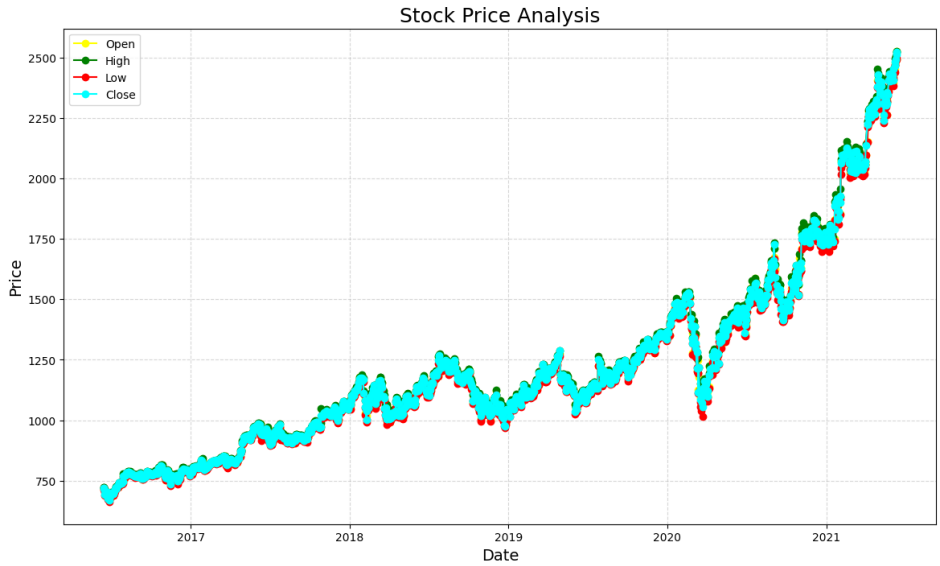

Figure 1: A plot of the 4 data sets provided in `GOOG.csv`

Figure 2: The closing stock price plotted against time

We then plotted the closing stock price alone to understand better the global and local patterns visible in the data. In Figure 2, we observe a general upward trend in the stock price, even though the data fluctuated heavily on smaller scales. We worried that the LTSM naively trained on this data would look at the general upward trend and continue it as a prediction rather than reflecting and understanding the fluctuations in the original dataset. This led us to limit the LSTM's "vision" to a smaller subset of data, looking at the last "X" data points rather than all available data. We hoped that a more focused LSTM would pick up on the day-to-day changes in stock price rather than the yearly trends.

Out of curiosity, we constructed a histogram (Figure 3) of daily price changes and observed an approximately normal distribution. This pattern suggests that the stock changes by a couple of dollars on most days, while significant price fluctuations are rare. This implies that the stock's price movements are generally stable and predictable, not heavily influenced by extreme events on a day-to-day basis as initially anticipated.

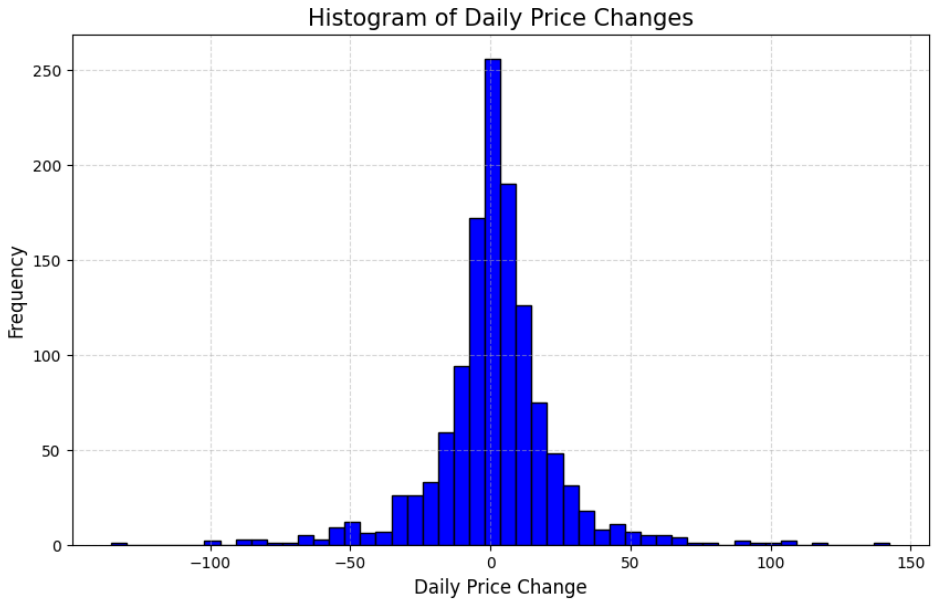

Figure 3: The closing stock price plotted against time

This finding reinforced our belief in using an LSTM model, known for its ability to discern causal patterns in time, especially when paired with data normalization. We noticed that stock prices from 2 adjacent days are highly correlated, while stock prices collected a year apart are almost completely uncorrelated (with a slight positive correlation accounting for the overall trends in data.) We hoped that an LSTM would be able to see both the local correlation in data through the noisy stock prices and the global correlation pushing the stock price up steadily.

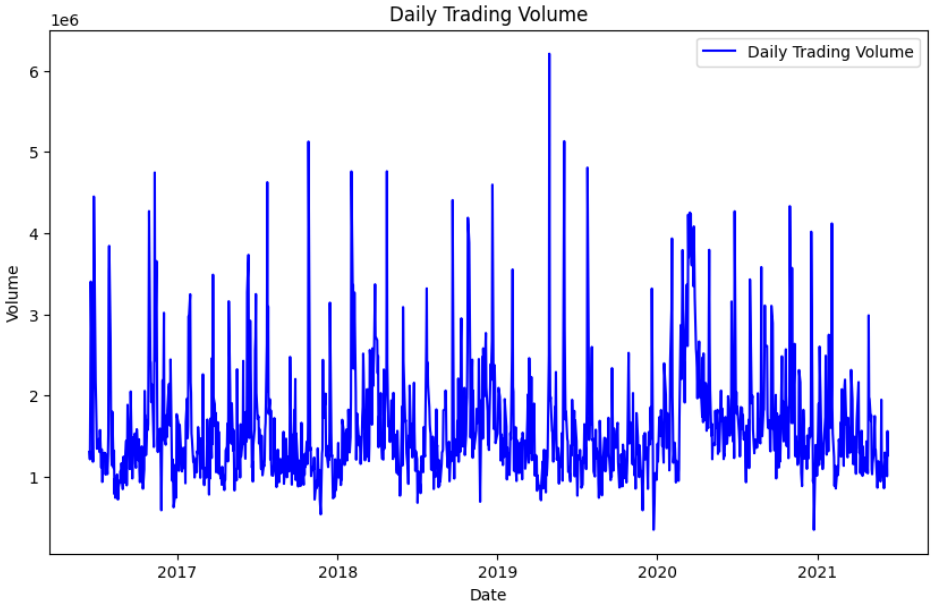

Figure 4: Plot of daily trading volume over time

We also sought to plot the daily trading volume. In Figure 4, each data point represents the number of shares traded that day. We hoped that high trading volume indicated high investor interest in selling or buying, indicating that the stock followed a strong directional trend. Analyzing the data, we found that high trading volume corresponds to stronger volatility, allowing the model to be more prepared for a larger bias in its prediction. Spikes in the graph might relate to news events or quarterly earnings

reports, indicating possible changes in the trends and patterns.

Similar to how humans learn, we hoped a modified LSTM could learn the correlation between this volatility and the current pattern to make more accurate predictions. Daily trading volume provides additional information that is not reflected in the stock price over time. Considering this information could give our model an edge in accurately predicting stock prices.

2.2 Data Preprocessing

The first step in our data preprocessing was identifying and removing missing values to maintain data integrity. Some data points in `GOOG.csv` were missing certain columns, and these data points were unusable. Luckily, we found only a few missing values in the data near the end, which we removed entirely from the dataset. Following this initial cleanup, we converted the 'date' column to a pandas datetime format and sorted the dataset chronologically. This ensured that all subsequent training steps and data cleaning maintained the correct sequence so that the time series data was in the correct order.

```
stock_data['date'] = pd.to_datetime(stock_data['date'])
stock_data = stock_data.sort_values('date')

stock = stock_data[['date', 'close', 'high', 'low', 'open', '
    volume']]
```

From our data exploration, we selected the most relevant columns for our analysis: the closing stock price and the time stamp. We then decided to normalize this data before feeding it to the model.

We normalized our data using Min-Max scaling, linearly mapping our data between 0 and 1, preventing the features at the extrema from overshadowing the data points in the middle. Our key insight that led to this decision was noticing that the change in the stock price carried more important information than the stock price itself: a stock increasing to 1200 from 1000 is a lower increase proportionally than that from 600 to 800, but the model should weight these two changes equally. Normalizing the data removes differences in numeric scales, allowing the LSTM to focus on and learn from underlying patterns in the data rather than being distracted by the numerical value of the input.

Normalization also had measurable benefits. In our own experiments, we also found that the normalized input training outperformed the unnormalized input training in speed to convergence, accuracy while training, and validation accuracy on the same test set. Empirically, we found the unnormalized model to be about 65% less accurate than the normalized model, proving that normalization was a crucial step in our preprocessing pipeline.

```
scaler = MinMaxScaler()
normalized_data = stock[['open', 'high', 'low', 'volume', '
    close']].copy()
normalized_data = scaler.fit_transform(normalized_data)
```

Next, we had to split our data into smaller subsequences for the model to learn, validate, and test. Building from our data exploration, we sought to limit the LSTM's attention to the 50 data points before the prediction. We used NumPy to extract the stock price column from `GOOG.csv`. Using array slicing and a sliding window, we split the dataset into the maximal number of 50-day periods. Once all the subsets were generated, we split the data into training, testing, and validation datasets.

```
def generate_sequences(df, seq_length=50):
    X = df[['open', 'high', 'low', 'volume', 'close']].
        reset_index(drop=True)
    y = df[['open', 'high', 'low', 'volume', 'close']].
        reset_index(drop=True)

    sequences = []
    labels = []

    for index in range(len(X) - seq_length + 1):
        sequences.append(X.iloc[index : index + seq_length].
            values)
        labels.append(y.iloc[index + seq_length - 1].values)

    sequences = np.array(sequences)
    labels = np.array(labels)

    return sequences, labels

train_validation_data, test_data = train_test_split(
    normalized_data, test_size=0.2, shuffle=False)
train_data, validation_data = train_test_split(
    train_validation_data, test_size=0.2, shuffle=False)

train_df = pd.DataFrame(train_data, columns=['open', 'high', '
    low', 'volume', 'close'])
val_df = pd.DataFrame(validation_data, columns=['open', 'high'
    , 'low', 'volume', 'close'])
test_df = pd.DataFrame(test_data, columns=['open', 'high', '
    low', 'volume', 'close'])

train_sequences, train_labels = generate_sequences(train_df)
val_sequences, val_labels = generate_sequences(val_df)
test_sequences, test_labels = generate_sequences(test_df)
```

As an aside, we did explore feature engineering and outlier removal but opted to retain outlier data based on our literature search. Removing outliers could bias our model towards accurately predicting less volatile, smoother patterns while potentially overlooking valuable insights into how stock market spikes and dips occur. By retaining these outliers, our LSTM model can learn from the variability and extreme events inherent in financial markets, enabling it to recognize and react to similar patterns in future data. This approach aligns with the LSTM's strength in capturing long-term dependencies and enhances its ability to provide robust predictions across varying market conditions.

We also decided to omit cross-fold validation as we had a fairly large dataset, especially once we generated the sequences of 50 using a sliding window. This allowed us to spend most of our time evaluating and ensuring the normalization step was valuable and preparing the data to try in different models to test and iterate.

2.3 Model Design

We decided to test various strategies to find a unique and effective model architecture to solve the problem. The first approach we explored was to examine current Kaggle solutions. As hypothesized, the most effective approach [Mir23] utilized an LSTM. Figure 5 shows the architecture of this model.

As depicted in Figure 6, the model followed this structure:

- A series of LSTM layers with a dropout layer in between to mitigate overfitting.
- The final dense layer reduced the output to a single value, which is the model's prediction.

This approach proved to be one of the best solutions on Kaggle. Drawing inspiration from this model format, we implemented our initial architecture, which adopted a similar approach but with a larger input dimension into the dense layer.

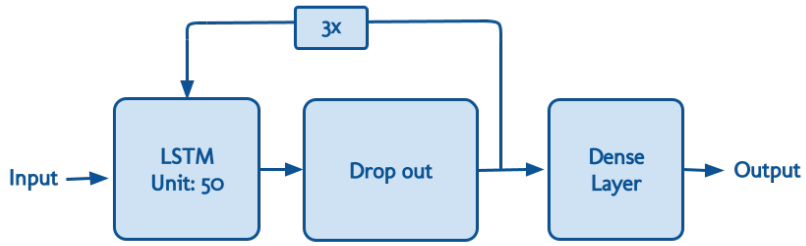

Figure 5: The architecture of a top performing Kaggle solution, using a simple LSTM

```
Model: "sequential"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 lstm (LSTM)                 (None, 50, 50)            11200

 dropout (Dropout)           (None, 50, 50)            0

 lstm_1 (LSTM)               (None, 50, 50)            20200

 dropout_1 (Dropout)         (None, 50, 50)            0

 lstm_2 (LSTM)               (None, 50)                20200

 dropout_2 (Dropout)         (None, 50)                0

 dense (Dense)               (None, 5)                 255

=================================================================
Total params: 51,855
Trainable params: 51,855
Non-trainable params: 0
_________________________________________________________________
```

Figure 6: A detailed breakdown of the model architecture

However, to further emphasize the importance of short-term features, we experimented with an architecture that featured parallel LSTM layers with varying input sizes: 50, 40, 30, 20, and 10 units, as shown in Figure 7. Despite training the model for 400 epochs, it consistently underperformed on the validation set due to severe overfitting. We attributed this issue to the repeated inputs in the LSTM stack, effectively nullifying the dropout layer.

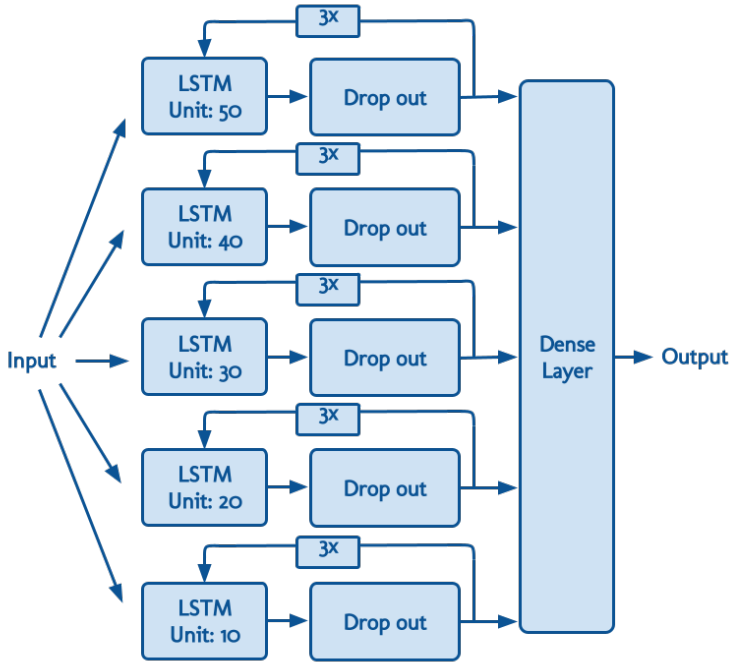

Figure 7: A diagram of our experimental Parallel LSTM architecture.

In a second attempt at this stacked LSTM architecture, we simplified by removing the 40 and 2 unit layers (Figure 8), which yielded marginally better but still unsatisfactory results as overfitting persisted. We attempted to alleviate this by incorporating batch normalization layers and increasing the dropout rate, but these adjustments yielded minimal improvement in overall performance.

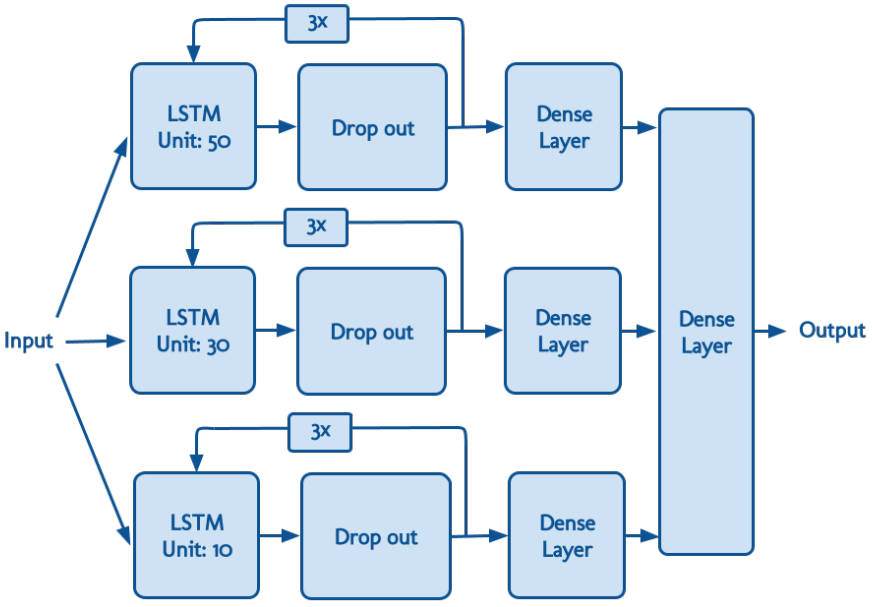

Figure 8: A diagram of our simplified experimental Parallel LSTM architecture.

To address these challenges, we restructured the model as depicted in Figure 9, opting for a BiLSTM (Bidirectional LSTM) architecture [SNTN19]. This model employs two LSTM layers: one processing data forward and the other backward. This approach effectively captures both data from the past and short-term dependencies in the data separately before combining them for prediction.

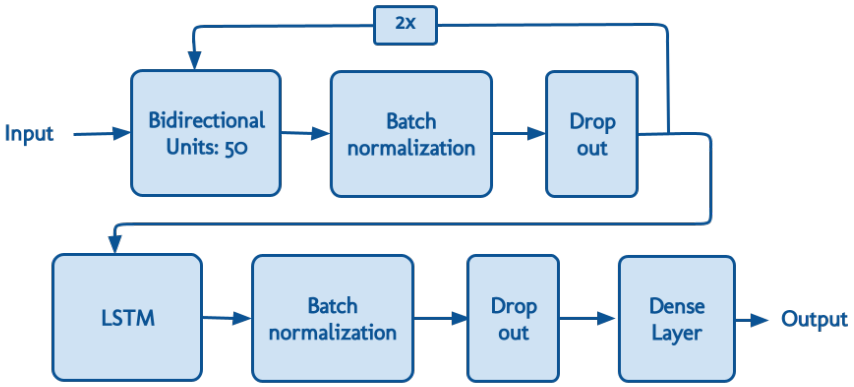

Figure 9: A diagram of our final BiLSTM architecture.

Unlike the earlier stacked LSTM model in Figure 7, where the model was exposed to the same data multiple times, we introduced a batch normalization layer before the dropout layer. We limited the BiLSTM loop to two passes, followed by a standard LSTM architecture. This modification aimed to prevent overfitting by diversifying the model's exposure to the data.

2.4 Training Strategy

We began our hyperparameter search with our finalized model architecture (Figure 10).

Model: "sequential"

Layer (type)	Output Shape	Param #
bidirectional (Bidirectional)	(None, 50, 128)	35,840
batch_normalization (BatchNormalization)	(None, 50, 128)	512
dropout (Dropout)	(None, 50, 128)	0
bidirectional_1 (Bidirectional)	(None, 50, 128)	98,816
batch_normalization_1 (BatchNormalization)	(None, 50, 128)	512
dropout_1 (Dropout)	(None, 50, 128)	0
lstm_2 (LSTM)	(None, 64)	49,408
batch_normalization_2 (BatchNormalization)	(None, 64)	256
dropout_2 (Dropout)	(None, 64)	0
dense (Dense)	(None, 5)	325

Total params: 185,669 (725.27 KB)

Trainable params: 185,029 (722.77 KB)

Non-trainable params: 640 (2.50 KB)

Figure 10: A diagram of our final BiLSTM architecture.

Using a systematic approach, we divided our hyperparameter search into multiple steps. Initially, we aimed to narrow down the range of our search due to the computational intensity of our model. Our first step involved identifying a rough estimate of where to focus our optimization efforts. To achieve this, we naively trained three models using different sets of hyperparameters to observe where approximate convergence occurred. Interestingly, all models began to converge around 150 epochs. Although this convergence point may not represent the most optimized model, we used 150 epochs as a baseline for our hyperparameter search. This approach balanced achieving decent model performance with keeping the search within a reasonable timeframe.

Additionally, as part of our hyperparameter tuning process, we conducted tests to determine the optimal learning rate for our model. Using a similar iterative approach, we systematically varied the learning rate across various values, from very small to relatively large. We found that the optimizer adjusted to the learning rate variations without significantly impacting overall model performance or convergence. This observation suggested that the adjustments being made by the Adam optimizer effectively addressed any potential issues with convergence. As a result, we decided to retain the default learning rate settings for subsequent experiments, focusing our efforts more intensely on regularization and architecture adjustments to enhance model robustness and performance. With this also came the decision to implement early stopping in our experiment. Knowing that the learning rate was optimized allowed us to realize that if the model was not converging after a reasonable amount of time or was "stuck," there was no point in continuing the experiment.

At this point, we focused on determining the optimal regularization rates for the three LSTM layers. Using a for loop, we systematically tested all combinations of regularization rates across the LSTMs. Figure 11 illustrates the results of this search:

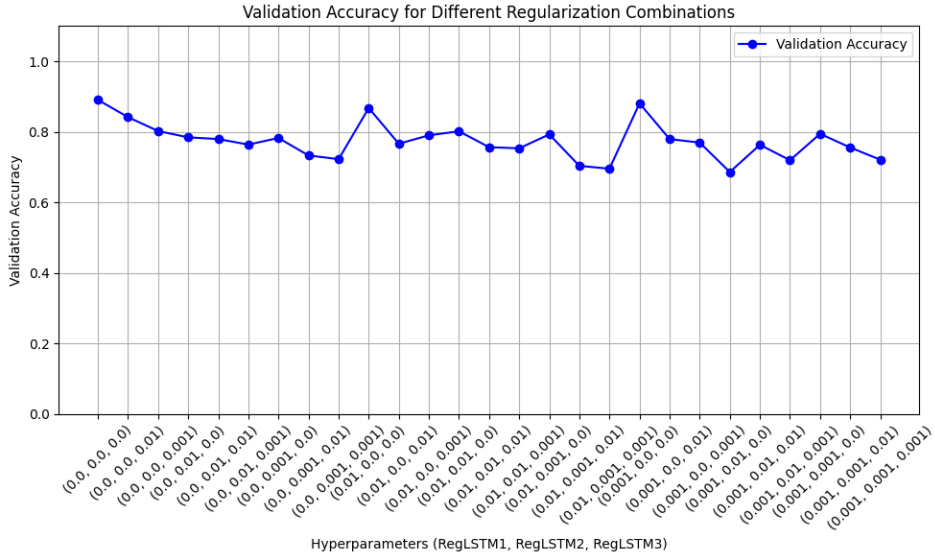

Figure 11: Validation accuracy for different regularization combinations

We identified the most effective regularization rates for each LSTM as 0.001, 0.0, and 0.0, respectively.

With the regularization terms set to their optimal values based on our earlier findings, the next step was to evaluate the impact of dropout rates on our model's performance. While using the optimal regularization rates, we systematically tested different dropout rates, batch sizes, and number of epochs to identify optimal settings across the model architecture. Figure 12, Figure 13 and Figure 14 below illustrates the normalized results obtained from this testing phase:

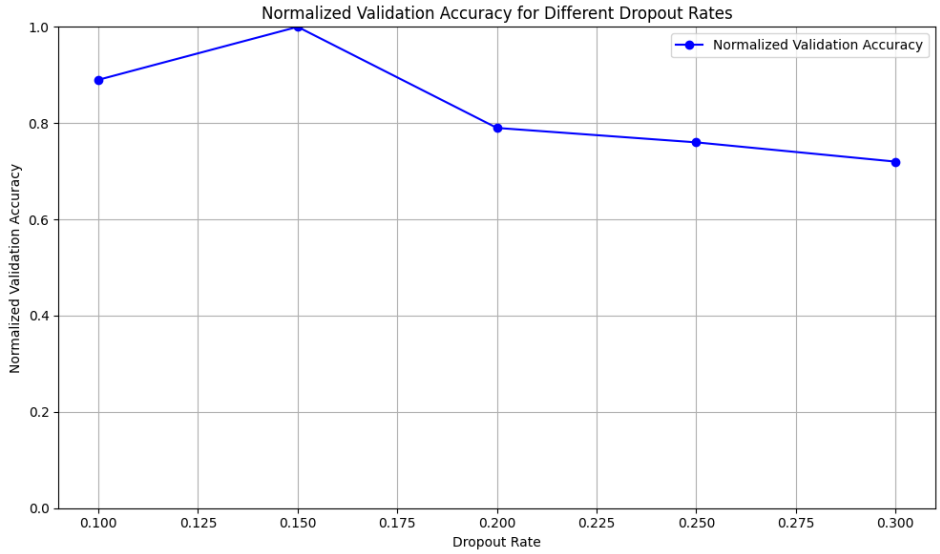

Figure 12: Normalized Validation Accuracy for different Dropout Rates

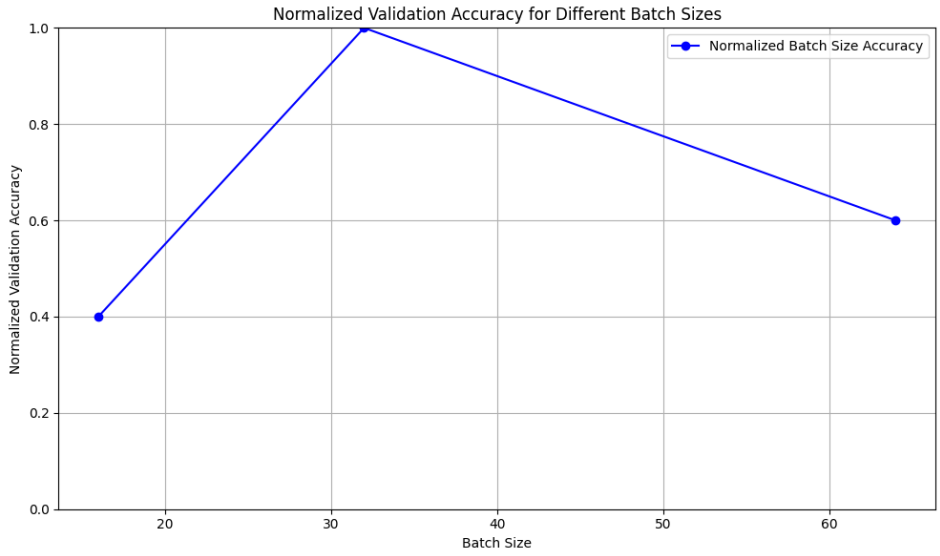

Figure 13: Normalized Validation Accuracy for Different Batch Sizes

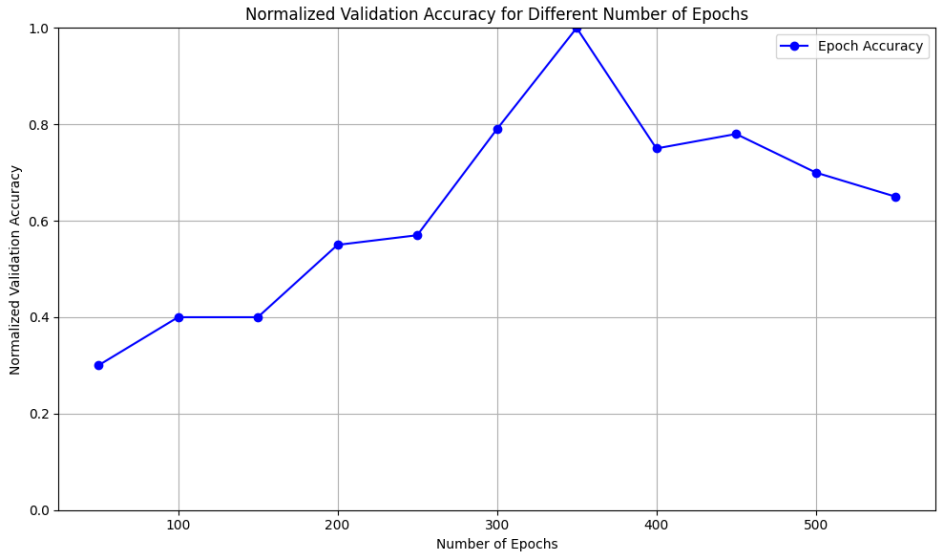

Figure 14: Normalized Validation Accuracy for Different Numbers of Epochs

From this experimentation, we identified that a dropout rate of 0.15 provided the best balance between preventing overfitting and retaining model performance. Additionally, a batch size of 32 and training for 350 epochs before overfitting occurred were optimal for our model. Furthermore, we refined our regularization terms to 0.001, 0.0, and 0.0, confirmed through iterative testing to enhance model generalization and performance.

3 Results

After using the hyperparameters found in the last section, we tested our model against the validation (Figure 15) and test datasets(Figure 16), and produced the following plots. Overall, the model's performance was solid. Our validation Mean Squared Error (MSE) was ≈ 0.000154, while our test MSE was ≈ 0.00259.

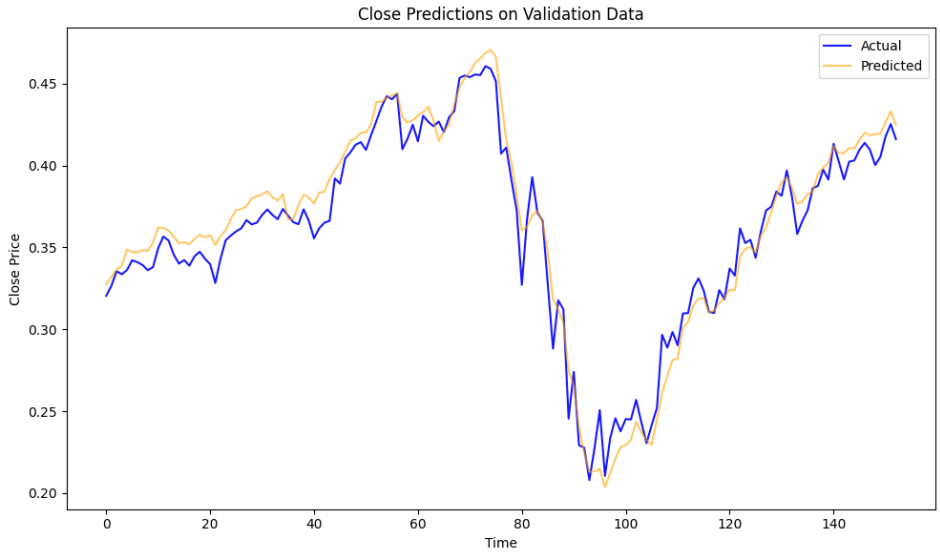

Figure 15: Close predictions on validation data

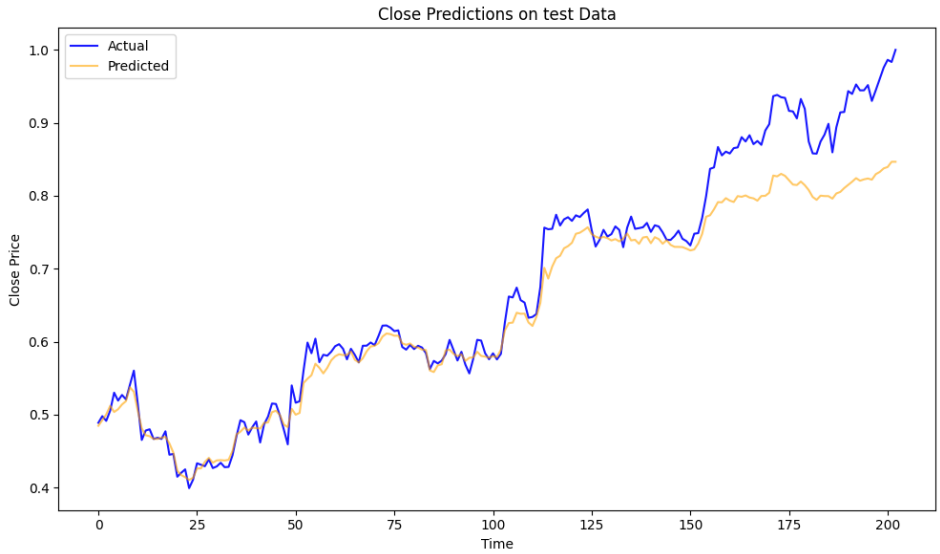

Figure 16: Close predictions on test data

This result underscores the effectiveness of our hyperparameter tuning process in enhancing model accuracy and generalization. The reduced mean square error reflects the model's improved ability to predict outcomes accurately across various timestamps, validating the efficacy of our approach in optimizing model performance.

4 Conclusion

This paper explored the prediction of stock prices using a deep neural network model based on a bidirectional Long Short-Term Memory (BiLSTM) architecture. Our approach involved experimentation with various model architectures and a rigorous hyperparameter search to optimize our model's performance.

The final BiLSTM model performed well in predictive tasks. This performance illustrates the potential of the BiLSTM architecture to provide valuable insights for financial analysis and decision-making. The model's ability to learn from historical data and predict future stock prices with high

accuracy can significantly benefit investors and financial analysts. Future work could explore integrating additional features such as sentiment analysis from news articles or real-time trading functionality for a greater grasp over the stock market.

5 Group Contributions

- Srinivasan Arumugham - Typesetting, Data Preprocessing, Literature Review
- Nikhil Kapasi - Programming, Architecture investigation, writing
- Rachit Gupta - Debugging, programming
- Riya Gupta - Programming, writing, initial architecture investigation
- Ethan Solomon - Programming, writing, diagrams

6 Model Notebook

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import MinMaxScaler
from sklearn.model_selection import train_test_split
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dropout, Dense, Bidirectional,
    BatchNormalization
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.regularizers import l2
from tensorflow.keras.callbacks import LearningRateScheduler
from plotly.subplots import make_subplots
import plotly.graph_objects as go
import plotly.io as pi
import plotly.express as px
import matplotlib.pyplot as plt
import datetime
from datetime import timedelta
```

```
from google.colab import files
uploaded = files.upload()
```

```
stock_data = pd.read_csv('/content/GOOG.csv')
```

```
stock_data['date'] = pd.to_datetime(stock_data['date'])
stock_data = stock_data.sort_values('date')
```

```
stock = stock_data[['date', 'close', 'high', 'low', 'open', 'volume']]
```

```
scaler = MinMaxScaler()
normalized_data = stock[['open', 'high', 'low', 'volume', 'close']].copy()
normalized_data = scaler.fit_transform(normalized_data)
```

```
train_validation_data, test_data = train_test_split(normalized_data, test_size
    =0.2, shuffle=False)
train_data, validation_data = train_test_split(train_validation_data,
    test_size=0.2, shuffle=False)
```

```
train_df = pd.DataFrame(train_data, columns=['open', 'high', 'low', 'volume',
    'close'])
validation_df = pd.DataFrame(validation_data, columns=['open', 'high', 'low',
    'volume', 'close'])
test_df = pd.DataFrame(test_data, columns=['open', 'high', 'low', 'volume', '
    close'])
```

```
def generate_sequences(df, seq_length=50):
    X = df[['open', 'high', 'low', 'volume', 'close']].reset_index(drop=True)
    y = df[['open', 'high', 'low', 'volume', 'close']].reset_index(drop=True)

    sequences = []
    labels = []

    for index in range(len(X) - seq_length + 1):
        sequences.append(X.iloc[index : index + seq_length].values)
        labels.append(y.iloc[index + seq_length - 1].values)

    sequences = np.array(sequences)
    labels = np.array(labels)

    return sequences, labels
```

```
train_sequences, train_labels = generate_sequences(train_df)
validation_sequences, validation_labels = generate_sequences(val_df)
test_sequences, test_labels = generate_sequences(test_df)
```

```
model = Sequential([
    Bidirectional(LSTM(units=64, return_sequences=True, kernel_regularizer=l2
        (0.001)), input_shape=(50, 5)),
    BatchNormalization(),
    Dropout(0.15),
    Bidirectional(LSTM(units=64, return_sequences=True, kernel_regularizer=l2
        (0.0))),
    BatchNormalization(),
    Dropout(0.15),
    LSTM(units=64, kernel_regularizer=l2(0.0)),
    BatchNormalization(),
    Dropout(0.15),
    Dense(units=5)
])
```

```
model.compile(loss='mean_squared_error', optimizer='adam', metrics=['
    mean_absolute_error'])
model.summary()
```

```
epochs = 350
batch_size = 32

history = model.fit(
    train_sequences,
    train_labels,
    epochs=epochs,
    batch_size=batch_size,
    validation_data=(validation_sequences, validation_labels),
    verbose=1
)
```

```
train_predictions = model.predict(train_sequences)
validation_predictions = model.predict(validation_sequences)
test_predictions = model.predict(test_sequences)
```

```
fig, ax = plt.subplots(figsize=(10, 6))

ax.plot(np.arange(len(test_labels[:, 0])), test_labels[:, 0], label='Actual',
    linestyle='-', color='blue', alpha=0.9)
ax.plot(np.arange(len(test_predictions[:, 0])), test_predictions[:, 0], label=
    'Predicted', linestyle='-', color='orange', alpha=0.6)

ax.set_title('Close Predictions on test Data')
ax.set_xlabel('Time')
ax.set_ylabel('Close Price')

ax.legend()

plt.tight_layout()
plt.show()

np.mean((test_labels[:, 0]-test_predictions[:, 0])**2)
```

References

[Mir23] AmirHossein Mirzaei. Google stock prediction: Lstm, Sep 2023.

[SNTN19] Sima Siami-Namini, Neda Tavakoli, and Akbar Siami Namin. The performance of lstm and bilstm in forecasting time series. In *2019 IEEE International Conference on Big Data (Big Data)*. IEEE, December 2019.